GitHub Expert Services

GitHub Copilot Tips, Tricks and Best practices

Agenda

- Welcome
- Introduction
- Best Practices
- Tips and Tricks
- FAQ
- Q & A
- Break
- Hands-on Lab

GitHub Copilot

Beyond code completion

It's no longer just a code completion tool in your editor—it now includes a chat interface that you can use in your IDE, a command line tool via CLI extension, a summary tool in your pull requests, a helper tool in your terminals, an orchestration tool of your MCP servers and agents and much, much more.

- Copilot

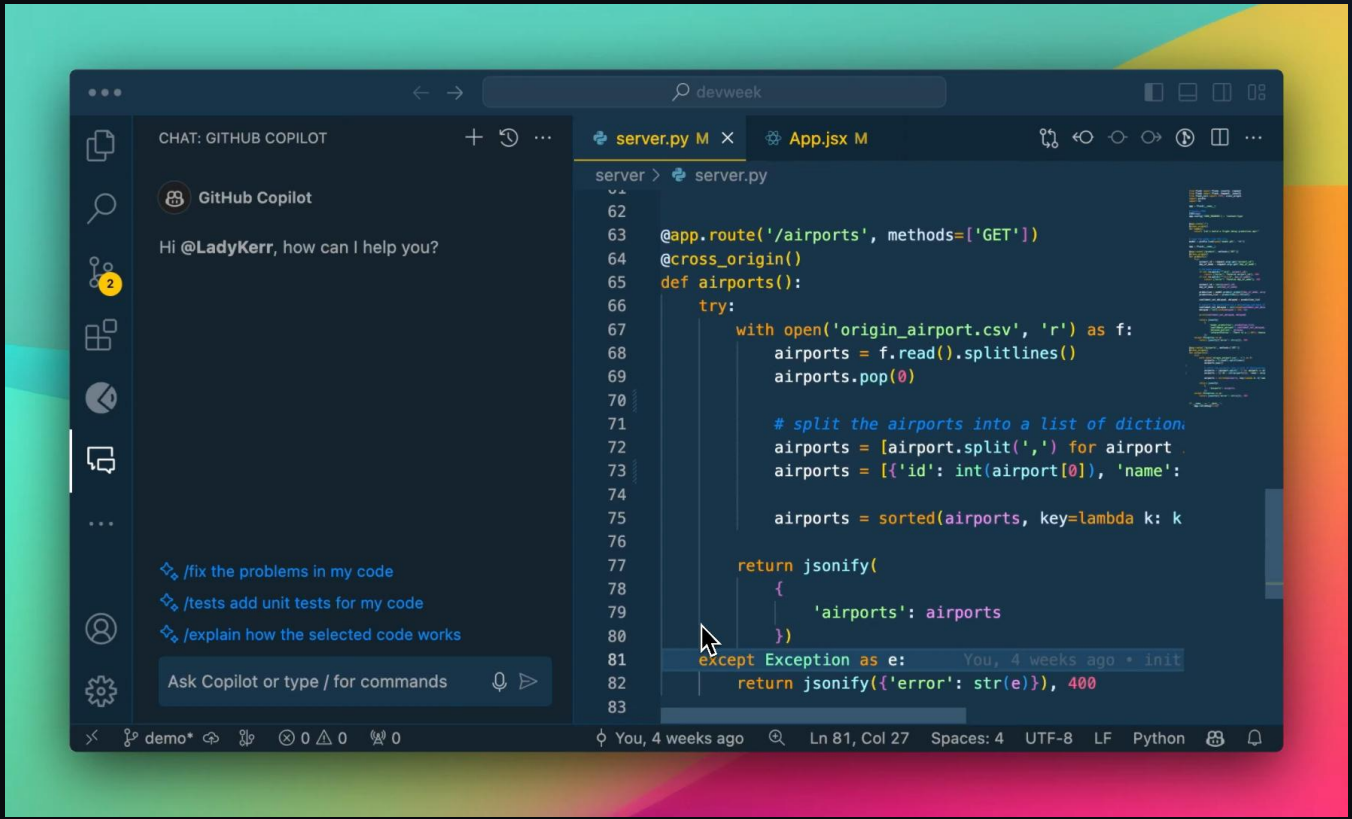
Context, Context, Context

- LLMs are designed to make prediction based on the context provided
- Provide ample context is the key
- Copilot is able to infer from the code in IDE editor
- How about adding more context?

Context, Context, Context

- Open Relevant files
- Provide Top level comment
- Set Includes and references

Context, Context, Context



Context, Context, Context

mealmetrics-copilot

CHAT: GITHUB COPILOT

GitHub Copilot

Hi @LadyKerr, how can I help you?

/fix the problems in my code

/tests add unit tests for my code

/explain how the selected code works

Ask Copilot or type / for commands

theme.js

utils > theme.js > theme > palette > secondary

You, 2 months ago | 1 author (You)

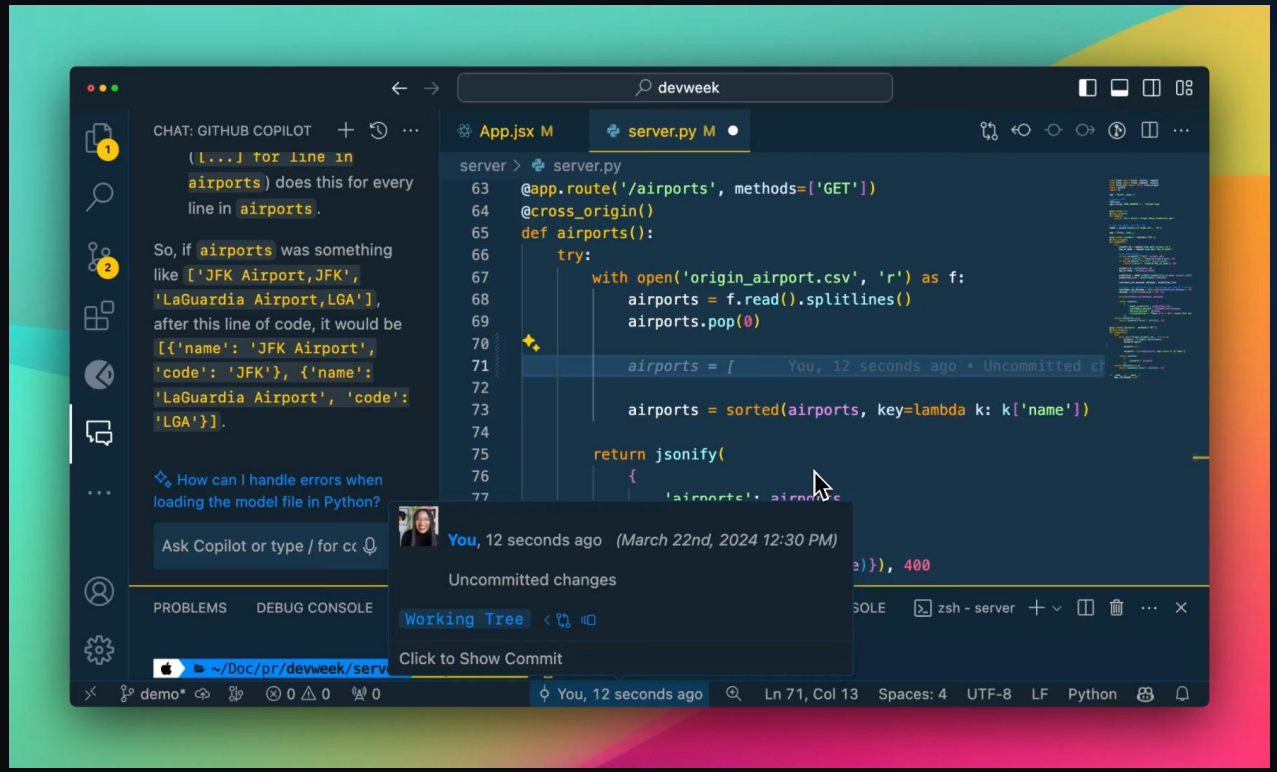
```
1  import { createTheme } from "@material-ui
2
3  /**
4   * Represents the theme object for the ap
5   *
6   * @typedef {Object} Theme
7   * @property {Object} palette - The color
8   * @property {Object} palette.primary - T
9   * @property {string} palette.primary.mai
10  * @property {Object} palette.secondary -
11  * @property {string} palette.secondary.m
12  */
13 const theme = createTheme({
14   palette: {
15     primary: {
16       main: "#ffc500",
17     },
18     secondary: {
19       main: "#f44336",        You, 2 months
20     },
21   },
22 });
23
```

demo 0↓ 2↑ 0 0 0 You, 2 months ago Ln 19, Col 23 Spaces: 2 UTF-8 LF {} JavaScript

Context, Context, Context

- Meaningful names
- Specific function comments
- Provide sample code
- Let’s give them a try

Context, Context, Context



Why Your Results Are Inconsistent

Survey Results from Mercury Team (18 responses)

The Reality:

❌ Only 11% getting "Mostly Yes" on desired outputs
❌ 89% dissatisfied with result consistency

Root Cause - Low Awareness:

- Context variables (#file, #selection, #editor) - 72% unaware
- Top-level comments - 83% unaware
- Conversation threads - 78% unaware
- Slash commands - 67% unaware

The Good News:

✓ These features exist in YOUR Copilot RIGHT NOW
✓ We're about to use them TOGETHER
✓ You'll be in the top 11% by end of session

Stop Using ChatGPT - Everything You Need Is Here

Mercury Survey Reality:
72% of you leave VS Code for ChatGPT/Claude

What You're Using External Tools For:
- "ChatGPT for more overall architectural conversations"
- "Brainstorming outside of an IDE"
- "Answering coding questions and generating documentation"

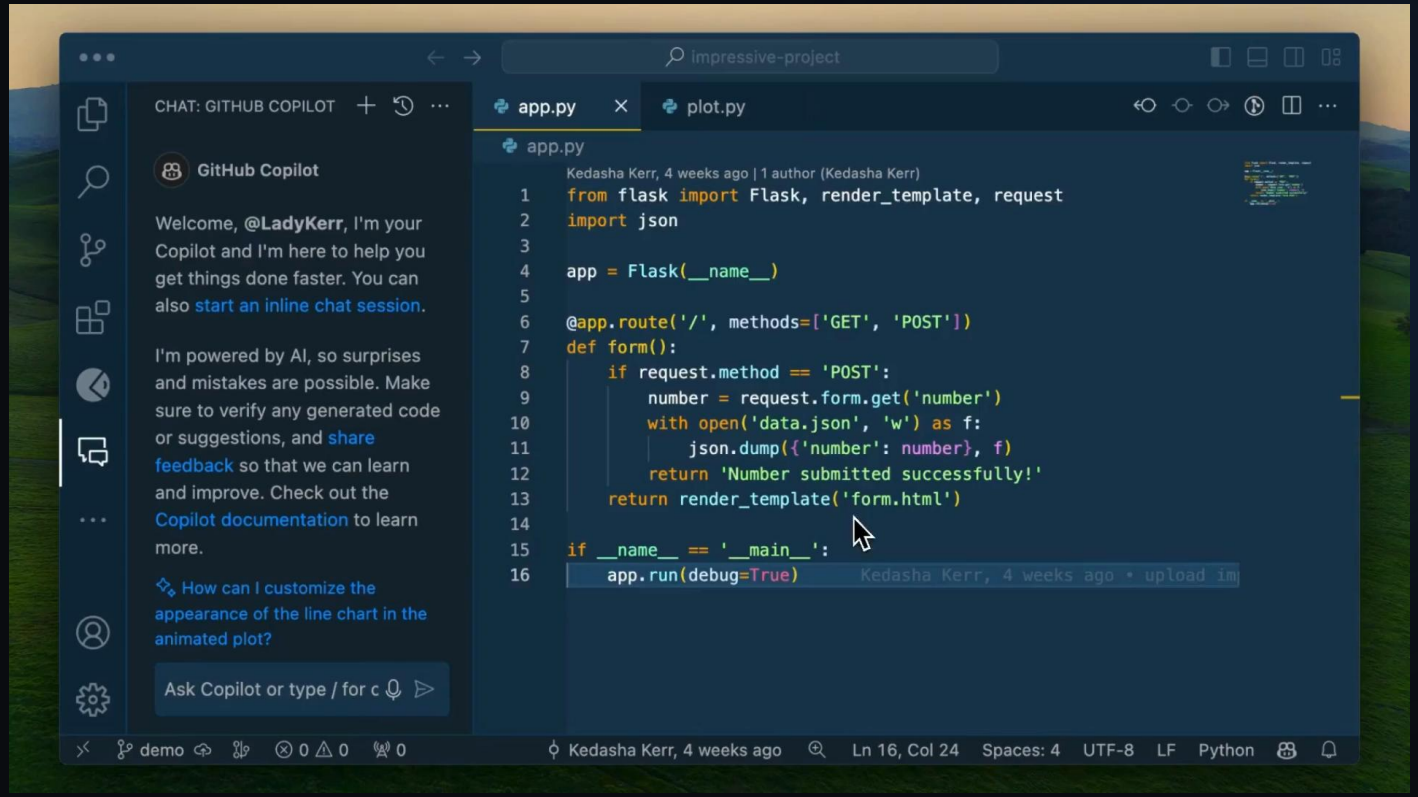
What You're Losing:
❌ Your codebase context
❌ File references
❌ Time switching windows
❌ Development flow
❌ Consistency across your team

What Copilot Chat Has That ChatGPT Doesn't:
✓ #codebase - knows YOUR Mercury codebase
✓ #file - references YOUR actual files
✓ /explain - works on YOUR project code
✓ Never leaves your IDE
✓ Remembers your conversation context

Inline Chat with GitHub Copilot

- CTRL + I on Windows
- Ask specific questions
- Works with #selection automatically
- See inline diffs before accepting
- Iterate without leaving your file
- Slash command support
- Pro Tip: Select code first, THEN Ctrl+I

Inline Chat with GitHub Copilot



GitHub Copilot CLI

- **You can use Copilot with the GitHub CLI to get started in any codebase instantly**
  #npm install copilot
- **Bring context from your issues and pull requests directly to your environment, eliminating context switching**
- **Build, edit, debug, and refactor code from your terminal**

GitHub Copilot CLI

```
C:\sources\MyMvcApp-Contact-Databse-Application (main -> origin)
λ
```

Copilot Chat

GitHub Copilot Chat provides an experience in your editor where you can have a conversation with the AI assistant. You can improve this experience by using built-in features to make the most out of it.

- Copilot

Copilot Chat

- Remove irrelevant requests
- Navigate through your conversation
- Use context variables: #file, #selection, #editor, #codebase
- Highlight relevant code
- Organize your conversations with threads

Role Prompting

Use copilot to assume a role during a task

Focus to improves results

Improves results by narrowing the AI's focus

Enhanced Results

abelberhane

I want you to act as a front end developer coach. I will provide some questions and the task that I have to complete and would like for you to walk me through the process before our sprint due date. This could involve offering advice on various topics, such as utilizing the react framework or handling API calls. My first request is to help setup my development environment.

GitHub Copilot

Sure, I can help you with that. What programming language and framework are you planning to use for your front-end development? Also, what operating system are you using?

AI Hallucinations

- Nature of the Model
- Fabricated Outputs
- Highly Creative but not always right
- Reliance on Developers
- Feedback Loop

Copilot Plan mode

In GitHub Copilot Chat, "plan mode" is an intelligent feature that activates when you give it a complex task. Instead of immediately trying to execute the request, Copilot first outlines a step-by-step plan for your approval.

- Copilot

Copilot Plan mode

- **Transparency:** See what Copilot will do before it acts.
- **Control:** Correct the plan to prevent unwanted changes.
- **Accuracy:** Breaking down tasks leads to better, more reliable results.
- **Collaboration:** Work with the AI as a partner to agree on the best approach.

Copilot Plan mode

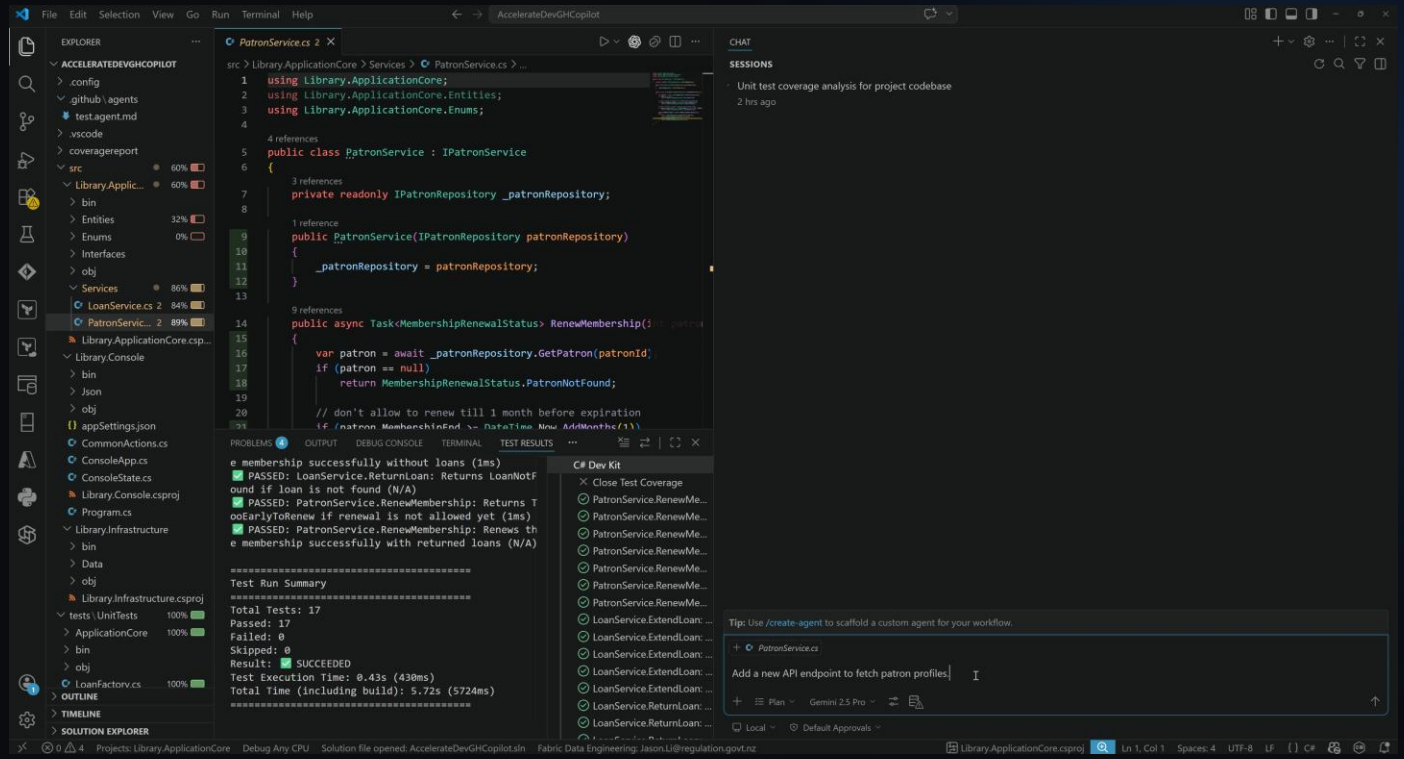
When to Use Plan Mode:

- Complex multi-file changes
- When you want to review before execution
- Learning what Copilot would do
- Building trust in AI suggestions

When NOT to Use:

- Simple, single-file edits
- Quick fixes or formatting
- When you're already confident in the approach

Copilot Plan mode



Copilot Agent Mode

In agent mode, Copilot Edits operates in a more autonomous and dynamic manner to achieve the desired outcome. Copilot agent mode determines the relevant context, offers both code changes and terminal commands, and iterates to remediate issues.

- Copilot

Copilot Agent Mode

- Copilot Edits Agent Mode uses a set of tools to accomplish the individual tasks to complete a request. E.g., listing the files in a directory, editing a file in your workspace, running a terminal command, interacting with MCP servers, and more.
- Still in Preview

Copilot Agent with MCP

File Edit Selection View Go Run Terminal Help

Welcome | settings.json | Untitled-1

C: > Users > jason > AppData > Roaming > Code - Insiders > User > settings.json > {} mcp

```
2   "mcp": {
11    "servers": {
12      "github": {
14        "args": [
15          "run",
16          "-i",
17          "--rm",
18          "-e",
19          "GITHUB_PERSONAL_ACCESS_TOKEN",
20          "ghcr.io/github/github-mcp-server"
21        ],
22        "env": {
23          "GITHUB_PERSONAL_ACCESS_TOKEN": "github_pat_11ABTKOEY0bDtDmqEXzl5s_GE3ErsHpATd2R
24        }
25      },
        Error | Restart
26      "time": {
27        "command": "docker",
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\jason>
```

CHAT

Edit with Copilot

Agent Mode

Ask Copilot to edit your files in agent mode. Copilot will automatically use multiple requests to pick files to edit, run terminal commands, and iterate on errors.

Copilot is powered by AI, so mistakes are possible. Review output carefully before use.

or type # to attach context

53 | Add Context... | settings.json Current file

Edit files in your workspace in agent mode

Agent | GPT-4.1 (Preview)

Ln 10, Col 7 | Spaces: 2 | UTF-8 | LF | JSON with Comments

How is Copilot Plan mode different from Copilot Agent mode?

Feature	Ask Mode	Plan Mode	Agent Mode
Core Idea	A general-purpose conversational assistant for Q&A and simple tasks.	A safety check for a single complex task.	A reusable expert for a specific role.
How to Use	**Default behavior.** Just type a question or a simple command.	**Automatic** for complex requests.	**Explicitly** call the agent with `@name`.
Customization	Not customizable.	Not customizable.	Highly customizable via `.agent.md` files.
Goal	Answering questions, explaining code, generating snippets.	Transparency and control.	Specialization and workflow automation.

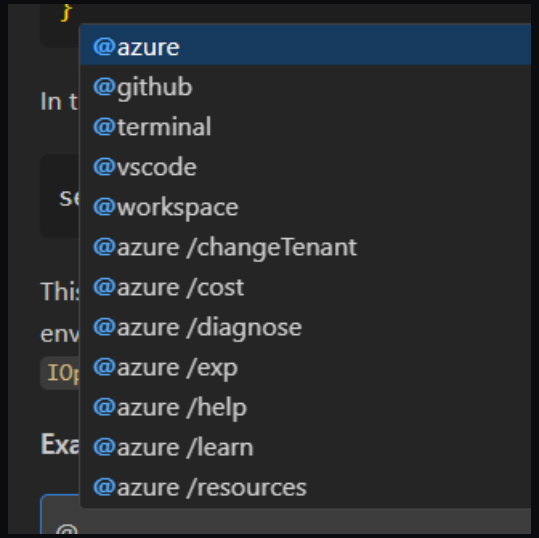
Slash Commands

Command	Description	Usage
/explain	Get code explanations	Open file with code or highlight code you want explained and type: /explain what is the fetchPrediction method?
/fix	Receive a proposed fix for the problems in the selected code	Highlight problematic code and type: /fix propose a fix for the problems in fetchAirports route
/tests	Generate unit tests for selected code	Open file with code or highlight code you want tests for and type: /tests
/help	Get help on using Copilot Chat	Type: /help what can you do?
/clear	Clear current conversation	Type: /clear
/doc	Add a documentation comment	Highlight code and type: /doc You can also press CMD+I in your editor and type /doc/ inline

Slash Commands

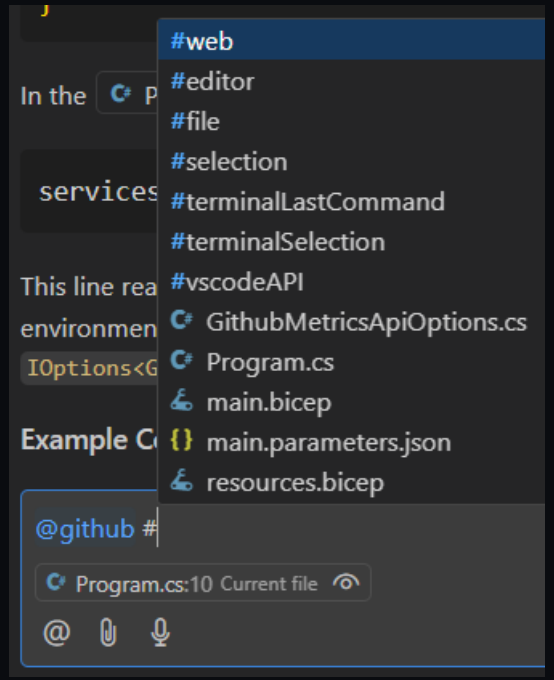
/generate	Generate code to answer your question	Type: /generate code that validates a phone number
/optimize	Analyze and improve running time of the selected code	Highlight code and type: /optimize fetchPrediction method
/clear	Clear current chat	Type: /clear
/new	Scaffold code for a new workspace	Type: /new create a new django app
/simplify	Simplify the selected code	Highlight code and type: /simplify
/feedback	Provide feedback to the team	Type: /feedback

Chat Participants (Custom Agents)



- Doesn’t respond straight away
- Analyzes the current workspace files and directories
- Creates a context from everything it was able to collect.

Chat Variables



- Use chat variables to include specific context in your prompt.
- Use # in the chat prompt box, followed by a chat variable.
- #webSearch is in GA that uses Bing web search engine

The Most Powerful Feature You're Not Using

#file:path/to/file
- Reference specific files for context
- "Explain how #file:LoanService.cs handles validation"

#selection
- Reference your highlighted code
- Select code, then: "Refactor #selection to use async/await"

#editor
- Reference your currently open file
- "Add error handling to #editor following Mercury standards"

#codebase
- Search across entire workspace
- "Find where we handle #codebase loan extensions"

HANDS-ON: Try Context Variables NOW

Open Your IDE and Complete These Tasks:

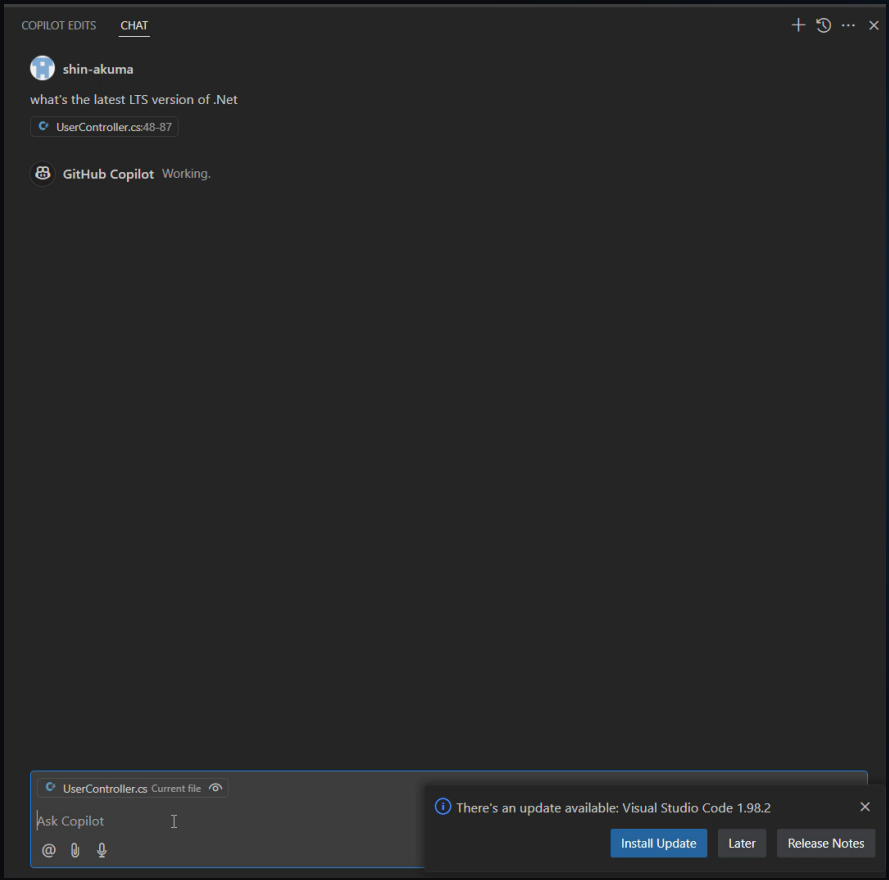
TASK 1: Type #file in Copilot Chat
→ Select any file from your project
→ Ask: "What does #file do?"

TASK 2: Select 5-10 lines of code
→ Open chat mode
→ Type: "Explain #selection in simple terms"

TASK 3: With a file open, ask:
→ "Add XML documentation to #editor"

TASK 4: Try @codebase
→ Ask: "Where is loan validation handled in @codebase?"

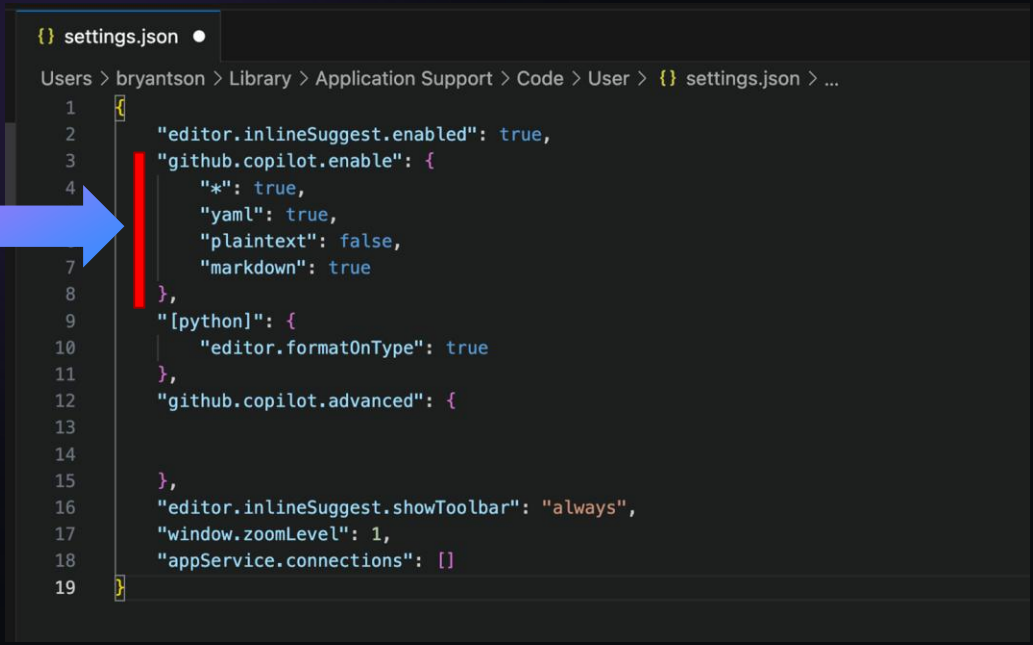
TASK 5: Create a conversation thread
→ Click “+" in chat → "New Thread"
→ Ask 3 related questions in sequence


COPILOT EDITS
CHAT
shin-akuma
what's the latest LTS version of .Net
UserController.cs:48-87
GitHub Copilot Working.
UserController.cs Current file
Ask Copilot
There's an update available: Visual Studio Code 1.98.2
Install Update
Later
Release Notes

Enable non-programming Meta Files

In **settings.json**, add

```
"github.copilot.enable":
{
  "*": true,
  "yaml": true,
  "plaintext": false,
  "markdown": true
}
```



Let’s think step by step

Breaking Down Tasks

Itemises each step for you

Ensuring Clarity

Takes time to spell everything out

Avoiding Overwhelm

Organised approach

abelberhane

Can you write the code for a calculator project?

GitHub Copilot

Sure! Here's an example of a simple calculator project in JavaScript:

```javascript
const calculator = {
  add: (a, b) => a + b,
  subtract: (a, b) => a - b,
  multiply: (a, b) => a * b,
  divide: (a, b) => {
    if (b === 0) {
      throw new Error('Division by zero');
    }
    return a / b;
  },
};

const calculate = (a, b, operator) => {
  switch (operator) {
    case '+':
      return calculator.add(a, b);
    case '-':
      return calculator.subtract(a, b);
    case '*':
      return calculator.multiply(a, b);
```

Let’s think step by step

Breaking Down Tasks

Itemises each step for you

Ensuring Clarity

Takes time to spell everything out

Avoiding Overwhelm

Organised approach

abelberhane

Can you write the code for a calculator project?

Let's think step by step.

GitHub Copilot

Sure! Here's a step-by-step plan for building a calculator project:

1. Create an HTML file with a basic layout for the calculator, including a display area and buttons for each digit and operation.
2. Add CSS styles to make the calculator look more visually appealing.
3. Create a JavaScript file to handle the calculator logic.
4. Define variables to store the current value, the previous value, and the current operation.
5. Add event listeners to the digit and operation buttons to update the current value and display it on the calculator.
6. Add event listeners to the operation buttons to perform the appropriate calculation and update the display with the result.
7. Add a clear button to reset the calculator to its initial state.
8. Test the calculator thoroughly to ensure that it works correctly for all inputs and operations.

Here's an example of what the JavaScript code might look like:

```
// Define variables
let currentValue = '';
let previousValue = '';
let currentOperation = null;

// Get DOM elements
const display = document.querySelector('.display');
```

More tips

- Attach relevant files for reference
- Start with GitHub Copilot Chat for faster debugging
- Be on the lookout for sparkles!

More tips

CHAT: GITHUB COPILOT

reader object which will iterate over lines in the specified csv file. Here's how you can do it:

```
import csv

@app.route('/airports', methods=['GET'
@cross_origin()
def airports():
    try:
        with open('origin_airport.csv'
            reader = csv.reader(f)
            next(reader, None)  # skip

            airports = [{'id': int(row
            airports = sorted(airports

        return jsonify(
            {
```

How can I install the CSV reader library in my Python project?

Ask Copilot or type / for commands

server.py M | App.jsx M

server > server.py

```
from flask import Flask, jsonify, request
from flask import Flask, request, jsonify
from flask_cors import CORS, cross_origin
import pickle
import re

app = Flask(__name__)

# Enable CORS
CORS(app)
app.config['CORS_HEADERS'] = 'Content-Type'


@app.route('/')
@cross_origin()
def home():
    return "Let's build a flight delay prediction api!"


# Load the model from the file
model = pickle.load(open('model.pkl', 'rb'))

app = Flask(__name__)
```

demo* | 0 0 | 0 | You, 4 weeks ago | Ln 62, Col 1 | Spaces: 4 | UTF-8 | LF | Python

Mercury Success Stories - What Your Teammates Did

GitHub Copilot

Real Results from Your Mercury Colleagues:

🏆 "Completed a full feature Jira ticket, with tests and docs without manually writing any code"
→ Used: Agent mode + Custom instructions

🏆 "How helpful it's been with helping me fix production issues... PowerShell scripts"
→ Used: Copilot Chat with context + /explain

🏆 "First time using agent to complete task - was amazed"
→ Used: Agent mode + Clear prompting

🏆 "Using Copilot while a solution is in debug mode and getting it to explain/fix issues"
→ Used: Context awareness + /fix command

Mercury Success Stories - What Your Teammates Did

GitHub Copilot

```
What Made Them Successful?
✓ They learned the advanced features (not just auto-complete)
✓ They provided context (#file, @codebase)
✓ They used the right tool for the job (slash commands, agents)

You can do this too. You have the same tools.
```

Conclusion

- Context
- Prompt Crafting
- Stay in control

Before You Leave - Success Checklist

✅ SUCCESS CHECK - Can You Do This?

- ☐ I know how to use #file or #selection in a prompt
- ☐ I've tried @codebase for codebase questions
- ☐ I've used at least ONE slash command (/explain, /fix, /tests)
- ☐ I understand conversation threads vs single questions
- ☐ I know when to stay in Copilot vs using external ChatGPT
- ☐ I'm committed to trying ONE new feature tomorrow

Checked All Boxes?
🎉 You're now in the TOP 20% of Copilot users!

Haven't Checked All?
📅 Book a 1:1 pairing session: jason.li@arinco.co.nz
💬 Join #group-github-accelerator Slack channel
📖 Review these slides: link and recording will be shared later today

Frequently Asked Questions

"Why am I not getting good results from Copilot?"

“Is the information shared with GitHub Copilot secure?”

“For example, let’s say I enter some sensitive information through GitHub Copilot, is it going to store the data like my user information?”

"Can I train GitHub Copilot on my private codebase?"

"Is the Copilot suggestion from an IDE based on my computer, a project opened in my IDE, files opened through tabs or just a single file?"

"What telemetry data can I get from GitHub Copilot ?"

“How can GitHub Copilot integrate with existing pipelines/automations/scripts that we have?

“For example, I have this GitHub Actions/Azure DevOps pipeline, can GitHub Copilot integrate with those products so our code will always be secure?”

“Will Copilot replace my job”

“How much of my work can it take over?”

Q & A

Analyze and document code using GitHub Copilot

- Analyze and document code using GitHub Copilot | mslearn-github-copilot-dev
- https://github.com/MicrosoftLearning/mslearn-github-copilot-dev/raw/refs/heads/main/DownloadableCodeProjects/Downloads/AZ2007LabAppM2.zip

Develop new code features using GitHub Copilot

- Develop new code features using GitHub Copilot | mslearn-github-copilot-dev
- https://github.com/MicrosoftLearning/mslearn-github-copilot-dev/raw/refs/heads/main/DownloadableCodeProjects/Downloads/AZ2007LabAppM3.zip

Create unit tests by using GitHub Copilot Chat

- Develop unit tests using GitHub Copilot | mslearn-github-copilot-dev
- https://github.com/MicrosoftLearning/mslearn-github-copilot-dev/raw/refs/heads/main/DownloadableCodeProjects/Downloads/AZ2007LabAppM4.zip

Challenge 1

- C#
- https://learn.microsoft.com/en-gb/training/modules/generate-documentation-using-github-copilot-tools/6-exercise-complete-code-documentation-challenge
- Python
- https://experience.cloudlabs.ai/#/labguidepreview/f9fd80ac-fc1a-4609-a3f1-06650aec389e (Page 9)

Challenge 2

- C#
- https://experience.cloudlabs.ai/#/labguidepreview/f9fd80ac-fc1a-4609-a3f1-06650aec389e (Page 6)
- Python
- https://experience.cloudlabs.ai/#/labguidepreview/f9fd80ac-fc1a-4609-a3f1-06650aec389e (Page 5)

Upcoming Sessions

1. Intro to Prompt Engineering
2. GitHub Copilot for Data/ML Engineers
3. GitHub Copilot for DevOps Engineers
4. GitHub Copilot for Developers

Thank you!